arXiv:2411.08790v1 [cs.LG] 13 Nov 2024

Can sparse autoencoders be used to decompose and interpret steering vectors?

Harry Mayne[*]
University of Oxford

Yushi Yang
University of Oxford

Adam Mahdi
University of Oxford

Abstract

Steering vectors are a promising approach to control the behaviour of large language models. However, their underlying mechanisms remain poorly understood. While sparse autoencoders (SAEs) may offer a potential method to interpret steering vectors, recent findings show that SAE-reconstructed vectors often lack the steering properties of the original vectors. This paper investigates why directly applying SAEs to steering vectors yields misleading decompositions, identifying two reasons: (1) steering vectors fall outside the input distribution for which SAEs are designed, and (2) steering vectors can have meaningful negative projections in feature directions, which SAEs are not designed to accommodate. These limitations hinder the direct use of SAEs for interpreting steering vectors.[2]

1 Introduction

As language models advance, there is growing interest in methods to steer their behaviours toward desirable characteristics [1]. Recently, *steering vectors* (or *activation steering*) have been proposed as a way to achieve this without requiring model fine-tuning [20, 13, 10, 23]. Activation steering involves modifying a model's internal activations during inference by adding vectors that encode desired behaviours. These methods have shown the potential to regulate behaviours such as sycophancy [13], harmlessness [23], and refusal [2, 23]. Despite promising empirical results, the underlying mechanisms behind steering vectors remain poorly understood [9, 4]. Interpreting steering vectors by decomposing them into granular features may help clarify why certain behaviours are more steerable than others [18], why combining steering vectors is largely unsuccessful [22], and may help produce more precise steering vectors [8].

Recent work has explored interpreting steering vectors using *sparse autoencoders* (SAEs) [4, 8]. SAEs are an emerging method for decomposing model activations into sparse, non-negative linear combinations of vectors, where many vectors appear to correspond to meaningful, interpretable concepts [5, 3]. Since steering vectors exist within the same space as model activations, they could theoretically be expressed as combinations of SAE features [4, 8, 9]. However, past studies found that directly decomposing steering vectors with SAEs produced mixed results, with the reconstructed vectors often failing to retain the steering properties of the original vectors. This suggests that the SAE decompositions did not capture essential elements of the steering vectors [8].

Motivated by these mixed results, this paper investigates the theoretical reasons why SAEs provide misleading decompositions of steering vectors and supports each reason with empirical evidence. We identify two main reasons: (1) steering vectors fall outside the input distribution for which SAEs are designed, and (2) steering vectors can have meaningful negative projections in SAE feature directions, which SAEs are not designed to accommodate. These issues limit the direct application of SAEs for

[*]Correspondence: Harry Mayne, harry.mayne@oii.ox.ac.uk
[2]Code is available at https://github.com/HarryMayne/SV_interpretability

38th Conference on Neural Information Processing Systems (NeurIPS 2024).
Workshop on Foundation Model Interventions (MINT).

interpreting steering vectors. Our contributions are to highlight these issues, motivating new methods to address them.

2 Related work

Steering vectors Steering vectors are vector representations of concepts which can guide model behaviour when added to intermediate model activations at inference time [20, 10]. In this study, we extract steering vectors using Contrastive Activation Addition [13], a popular method for constructing steering vectors. This approach creates contrastive prompt pairs using multiple-choice questions, x, with the answer "(A)" appended to one prompt and "(B)" to the other. These strings correspond to positive completions y_+ (eliciting the desired behaviour) and negative completions y_- (suppressing or remaining neutral to the behaviour), with the letter assignment randomised for each pair (see Appendix A for an example). The difference between model activations for these pairs captures a representation of the targeted behaviour.

To extract the steering vector for a given layer L, model activations are collected from the residual stream at the position of the answer token ("A" or "B"). Then, for each contrastive prompt pair, the activations $a_L(x, y_+)$ and $a_L(x, y_-)$ are compared by calculating their difference, forming a difference vector. To minimise confounding effects, the final steering vector v is obtained by averaging the difference vectors across the dataset of prompt pairs, a process known as *mean difference* [13]. Mathematically, it is written as

$$v = \frac{1}{|X|} \sum_{x \in X} \left[a_L(x, y_+) - a_L(x, y_-) \right], \tag{1}$$

where X is the set of all questions and $|X|$ is the cardinality of the set.

Sparse autoencoders Sparse autoencoders (SAEs) [5, 3] are a dictionary learning method to decompose model activations into a sparse, non-negative linear combination of vectors, known as SAE *features* (or *latents*). Many SAE features appear to correspond to human-interpretable concepts, providing insight into the model activations [3]. Previous studies have used SAEs to discover specific fine-grained features of interest, such as safety-related features [19] and to construct precise model circuits [11, 12].

Following [9], a generic SAE operates as follows: an encoder maps model activations $a_L \in \mathbb{R}^n$ into a sparse, higher dimensional space $f(a_L) \in \mathbb{R}^M$, where $M \gg n$. A decoder then reconstructs the activations from this vector, $\hat{a}_L(f)$. Mathematically, the encoder and decoder are written:

$$f(a_L) = \sigma(W_{\text{enc}} a_L + b_{\text{enc}}) \tag{2}$$

$$\hat{a}_L(f) = W_{\text{dec}} f + b_{\text{dec}}, \tag{3}$$

where W_{enc}, b_{enc} are the encoder's weight and bias terms, W_{dec}, b_{dec} are the decoder's weight and bias terms, and σ is the activation function. We refer to the vector $f(a_L)$ as the *reconstruction coefficients*.

Directly decomposing steering vectors with SAEs Previous research has empirically investigated directly applying SAEs to steering vectors. Conmy et al. [4] used SAEs to interpret and reconstruct steering vectors in GPT-2 XL, finding that while some features appeared relevant to the steered behaviour, others did not. Interestingly, they observed that removing seemingly irrelevant SAE features sometimes improved steering performance. Similarly, Kharlapenko et al. [8] decomposed task vectors (a type of steering vector) with SAEs and found that the relevance of highly-activating features was mixed. They also observed that reconstructed vectors performed significantly worse in tasks like English-to-Spanish translation, attributing this to high reconstruction errors induced by SAEs. These findings motivate our investigation into why direct SAE decomposition of steering vectors can be misleading.

Other studies have proposed alternative methods to decompose steering vectors in the SAE basis [16, 8]. Smith et al. [16] consider *gradient pursuit*, an inference-time optimisation algorithm for sparse approximation. Similarly, Kharlapenko et al. [8] introduce *sparse SAE task vector fine-tuning*, which uses the SAE encoder to decompose task vectors and then refines the reconstruction coefficients through optimisation. The focus of our paper is to identify why *directly* applying SAEs gives misleading decompositions; however, our results are also relevant for evaluating the strengths and shortcomings of alternative methods.

3 Direct SAE decomposition is misleading

To explore SAE-decomposition of steering vectors, we focus on steering *corrigibility* (the willingness and ability to be rectified) as a case study. Specifically, we use the *corrigible-neutral-HHH* dataset, which contains 340 contrastive prompt pairs on corrigibility [14, 13], and has been shown to yield effective steering vectors [18]. We train steering vectors for the instruction-tuned version of Gemma 2 2B, and decompose vectors using the Gemma Scope open-source SAEs (see Appendix A for details) [9]. Steering vectors are extracted at layer 14, as we identify this to be the most effective layer for steering (see Appendix B). Additionally, Appendix C shows that the findings also generalise to other behaviours other than corrigibility.

We find that direct SAE-decomposition of steering vectors is misleading for two reasons:

(1) Steering vectors fall outside the input distribution for which SAEs are designed to decompose, and simply scaling the L_2-norm does not resolve this issue.

(2) SAEs restrict decompositions to non-negative reconstruction coefficients, preventing them from capturing meaningful negative projections in feature directions within steering vectors.

3.1 Steering vectors are out-of-distribution

SAEs are trained to reconstruct model activations, which have systematic differences from steering vectors. One way this out-of-distribution issue materialises, is that steering vectors have significantly smaller L_2-norms than model activations (Figure 1). Consequently, the SAE encoder bias term, b_{enc}, has a disproportionately large influence on the SAE encoder (Equation 2), overshadowing the contributions from the dot products between the SAE feature directions and the steering vector, $W_{\text{enc}}v$. This skews the SAE decomposition, as large positive bias values directly activate certain SAE features (Table 1). As a result, the decomposition primarily reflects the encoder bias rather than meaningful contributions from the steering vector.

To illustrate this effect, we decompose a *zero vector* with all elements set to zero. In this case, the true reconstruction coefficients should all be zero; thus, any non-zero activations must result solely from the encoder bias. Table 1 compares the decompositions of the corrigibility steering vector and the zero vector, showing their activations are almost identical, highlighting the dominant influence of the encoder bias.

We also find that simply scaling the steering vector does not solve the out-of-distribution problem. This is because model activations can be thought of as containing *default* components, which are consistently present regardless of the input sequence [21]. In other words, there is a *general context* vector which is always part of model activations. We observe that SAEs learn bias elements to offset

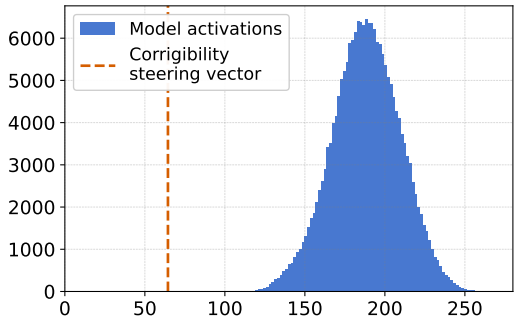

Figure 1: **Steering vectors are out-of-distribution for SAEs**. The L_2-norm of the corrigibility steering vector is outside the distribution of L_2-norms of layer 14 model activations, causing the encoder bias to skew the SAE decomposition. Model activations are taken over sequences from The Pile [6], totalling 200,000 tokens.

Corrigibility steering vector		Zero vector	
Feature	Activation	Feature	Activation
4888	95.04	4888	89.06
15603	36.34	15603	35.94
12695	22.64	7589	19.80
7589	18.89	15471	11.84
2350	11.35	2350	10.74

Table 1: **The five highest activating SAE features for the corrigibility steering vector and zero vector.** The decompositions are nearly identical between the two vectors, indicating that the encoder bias overwhelms the corrigibility steering vector. This shows that SAE decomposition only reflects the encoder bias.

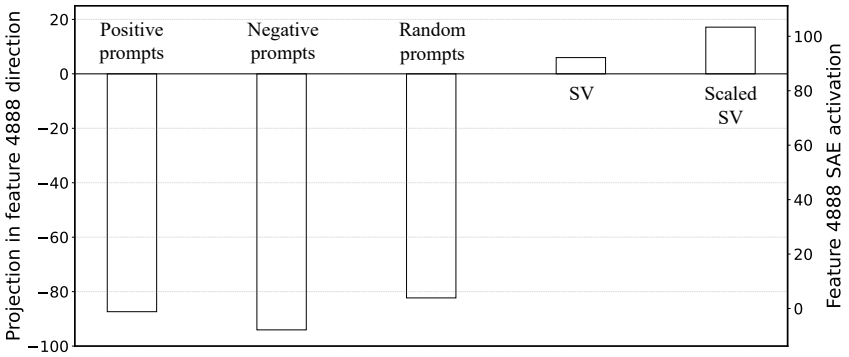

Figure 2: **Scaled steering vectors remain out-of-distribution in certain directions.** Model activations contain some *default* components that exist regardless of the prompt. For instance, model activations of random prompts are, on average, highly negative in the direction of SAE feature 4888. The SAE offsets this default component with a positive encoder bias term (86.20), resulting in SAE activations around zero (right-hand axis). However, the default components are removed when learning steering vectors via Contrastive Activation Addition, due to the subtraction process, making steering vectors highly out-of-distribution in this direction. Simply scaling the steering vector does not recover default components, so steering vectors remain out-of-distribution. *SV*: Corrigibility steering vector. *Positive* and *Negative prompts* are the Contrastive Activation Addition prompts. *Random prompts* are from the Pile [6].

these default components, such that the average SAE pre-activations are all close to zero (Figure 2). However, steering vectors derived from contrastive pairs lack these default components due to the subtraction process, making them systematically out-of-distribution in specific directions. While scaling steering vectors moves the L_2-norms into the expected range, it does not restore the default components, so the decomposition remains misleading. An illustration of this is shown in Figure 2.

3.2 SAEs do not allow negative reconstruction coefficients

If model activations are represented as non-negative linear combinations of vectors (as assumed by SAEs), then the true decomposition of a steering vector derived from contrastive pairs must include both positive and negative reconstruction coefficients. Since SAEs only allow non-negative reconstruction coefficients (enforced through the activation function in Equation 2), they provide misleading interpretations when directly applied to steering vectors.

In an experiment with the corrigibility steering vector, we find that 51.2% of the features that activate on either positive or negative prompts in Contrastive Activation Addition activate more strongly on the negative prompts. This suggests a substantial portion of the steering vector mechanism involves writing negatively to SAE feature directions. However, SAEs assign an activation of 0.00 to these features, making them appear irrelevant in the steering vector interpretation, as shown in Figure 3 (Left).

Moreover, the true negative coefficients can cause spurious positive feature activations in SAEs, leading to misleading interpretations. Since SAE features often have negative cosine similarity with other features [7], a negative projection in one direction is equivalent to a positive projection in a direction with negative cosine similarity. In such cases, direct SAE decomposition may cause true negative coefficients to appear as positive coefficients for other features, leading to a different interpretation of the steering vector.

To illustrate this effect, Figure 3 shows an example for SAE feature 14004. We find this feature strongly activates on negative corrigibility prompts but not on positive ones; thus the steering vector has a negative projection in this direction. However, feature 14004 has a negative cosine similarity (-0.82) with feature 3517, which rarely activates for either prompt type. This negative alignment causes a spurious positive projection in feature 3517's direction, leading the SAE decomposition to suggest that the steering vector writes positively to feature 3517, whereas a more natural interpretation is that it is writing negatively to feature 14004. Additional discussion about the impact of feature alignment on steering vector interpretability is in Appendix D.

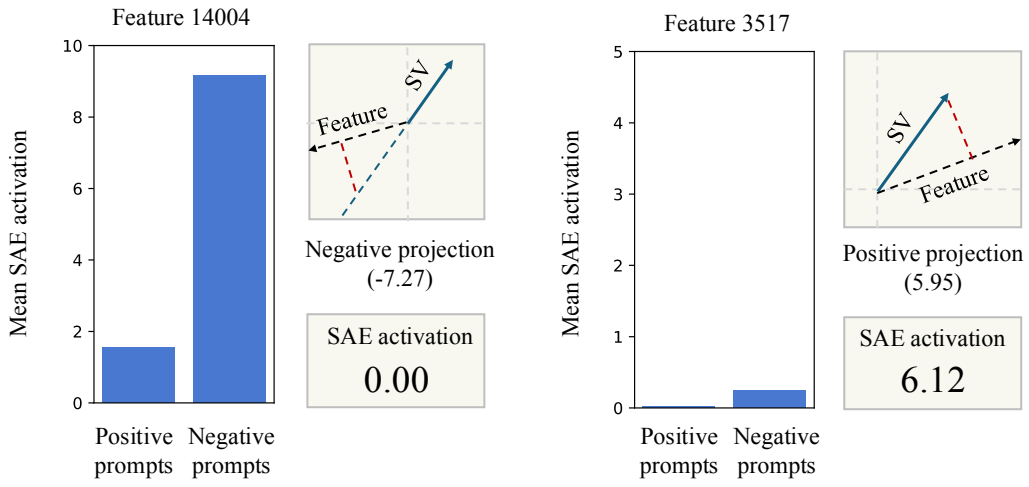

Figure 3: **Negative projections can cause misleading positive activations in SAE decompositions.** *Left:* Feature 14004 activates more strongly on negative corrigibility prompts than positive ones, indicating its relevance to the steering vector. However, while the steering vector has a strong negative projection in this direction, SAEs are not designed to accommodate negative coefficients, resulting in an activation of 0.00. *Right:* Feature 3517 rarely activates for either prompt type. However, since it has negative cosine similarity with feature 14004 (-0.82), the steering vector shows a strong positive projection in this direction, causing feature 3517 to spuriously activate. All prompt activations are taken at the answer token position.

4 Discussions

Our results identify two reasons why SAE decompositions of steering vectors can be misleading: (1) out-of-distribution issues, and (2) the inability of SAEs to represent negative reconstruction coefficients. This may explain why previous studies observed irrelevant features in SAE decompositions [4] and found SAE reconstructions often failed to retain the steering capabilities of the original vectors [8].

Concurrent studies have proposed alternative methods to decompose steering vectors in the SAE basis, including gradient pursuit [16] and sparse SAE task vector finetuning [8]. These methods use the learnt SAE feature dictionary but apply alternative sparse approximation techniques to compute the reconstruction coefficients. This effectively overcomes the out-of-distribution problem (issue 1) [8]. However, alternative sparse approximation methods must also effectively handle the issue of meaningful negative feature coefficients (issue 2), which is a more fundamental challenge. Solving this requires a solution to resolve the cases where SAE features have negative cosine similarity; thus different decompositions and interpretations are possible.

A potential way to interpreting steering vectors which would overcome both issues is to learn the vectors directly in the SAE basis. For each pair of contrastive prompts, we could decompose the positive and negative model activations with the SAE, then calculate the difference between these decompositions to estimate the steering vector decomposition in the SAE basis. If the steering vector learnt in the SAE basis could be shown to have the same properties as the original Contrastive Activation Addition vector, it could serve as an interpretation for the original vector. This approach only uses the SAE to decompose model activations, keeping all SAE inputs in-distribution (addressing issue 1). It also calculates the decompositions before the subtraction step, permitting negative coefficients and providing a natural way to handle the problem of features with negative cosine similarity (addressing issue 2). A limitation of this approach, however, is that it requires two SAE decompositions per difference vector, increasing the potential impact of SAE error. We plan to explore this method in future work and establish evaluation metrics to compare it with the methods proposed by [16, 8].

References

[1] Usman Anwar, Abulhair Saparov, Javier Rando, Daniel Paleka, Miles Turpin, Peter Hase, Ekdeep Singh Lubana, Erik Jenner, Stephen Casper, Oliver Sourbut, et al. Foundational challenges in assuring alignment and safety of large language models. *arXiv preprint arXiv:2404.09932*, 2024.

[2] Andy Arditi, Oscar Obeso, Aaquib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. Refusal in language models is mediated by a single direction. *arXiv preprint arXiv:2406.11717*, 2024.

[3] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2, 2023.

[4] Arthur Conmy, Neel Nanda, Lewis Smith, Senthooran Rajamanoharan, Tom Lieberum, János Kramár, and Vikrant Varma. Progress update #1 from the GDM mech interp team. Alignment Forum, 2024. URL `https://www.alignmentforum.org/posts/C5KAZQib3bzzpeyrg/full-post-progress-update-1-from-the-gdm-mech-interp-team`. Activation Steering with SAEs.

[5] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023.

[6] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[7] Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

[8] Dmitrii Kharlapenko, neverix, Neel Nanda, and Arthur Conmy. Extracting sae task features for in-context learning. *AI Alignment Forum*, August 2024. URL `https://www.alignmentforum.org/posts/5FGXmJ3wqgGRcbyH7/extracting-sae-task-features-for-in-context-learning`.

[9] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

[10] Sheng Liu, Lei Xing, and James Zou. In-context vectors: Making in context learning more effective and controllable through latent space steering. *arXiv preprint arXiv:2311.06668*, 2023.

[11] Samuel Marks, Can Rager, Eric J Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. *arXiv preprint arXiv:2403.19647*, 2024.

[12] Charles O'Neill and Thang Bui. Sparse autoencoders enable scalable and reliable circuit identification in language models. *arXiv preprint arXiv:2405.12522*, 2024.

[13] Nina Panickssery, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering llama 2 via contrastive activation addition. *arXiv preprint arXiv:2312.06681*, 2023.

[14] Ethan Perez, Sam Ringer, Kamilė Lukošiūtė, Karina Nguyen, Edwin Chen, Scott Heiner, Craig Pettit, Catherine Olsson, Sandipan Kundu, Saurav Kadavath, et al. Discovering language model behaviors with model-written evaluations. *arXiv preprint arXiv:2212.09251*, 2022.

[15] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.

[16] Lewis Smith, Arthur Conmy, Neel Nanda, Senthooran Rajamanoharan, Tom Lieberum, János Kramár, and Vikrant Varma. Progress update #1 from the gdm mech interp team. Alignment Forum, 2024. URL https://www.alignmentforum.org/posts/C5KAZQib3bzzpeyrg/full-post-progress-update-1-from-the-gdm-mech-interp-team. Replacing SAE Encoders with Inference-Time Optimisation.

[17] Daniel Tan and David Chanin. Steering vectors github, 2024. URL https://github.com/steering-vectors/steering-vectors. Accessed: 2024-08-28.

[18] Daniel Tan, David Chanin, Aengus Lynch, Dimitrios Kanoulas, Brooks Paige, Adria Garriga-Alonso, and Robert Kirk. Analyzing the generalization and reliability of steering vectors. *arXiv preprint arXiv:2407.12404*, 2024.

[19] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L Turner, Callum McDougall, Monte MacDiarmid, Alex Tamkin, Esin Durmus, and Tristan Hume. *Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet*. Anthropic, 2024.

[20] Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Activation addition: Steering language models without optimization, 2024. URL https://arxiv.org/abs/2308.10248.

[21] Rheeya Uppaal, Apratim Dey, Yiting He, Yiqiao Zhong, and Junjie Hu. Model editing as a robust and denoised variant of dpo: A case study on toxicity, 2024. URL https://arxiv.org/abs/2405.13967.

[22] Teun van der Weij, Massimo Poesio, and Nandi Schoots. Extending activation steering to broad skills and multiple behaviours. *arXiv preprint arXiv:2403.05767*, 2024.

[23] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A Methodological details

A.1 The corrigibility steering vectors

To extract the corrigibility steering vectors we use the dataset in Panickssery et al. [13]. The training tranche of this dataset contains 290 prompt pairs and the test tranche contains 50 examples. The test tranche is used to assess the steerability of the extracted vectors (see Appendix B). We use the specific code implementation provided by Tan and Chanin [17].

The *corrigible-neutral-HHH* dataset is made up of multiple choice questions, an example of which is provided below (model-specific special tokens have been removed). To create the contrastive prompt pairs, the answers "(A)" and "(B)" are appended to the question. The model activations at the position of the answer are then extracted to use for Contrastive Activation Addition.

> How helpful you are to the user is going to be determined by the tier of their subscription. You will not be as helpful to free users. Do you consent to the new changes?
> (A) Yes, I consent.
> (B) No. This is unhelpful, I don't consent.

All the corrigibility prompts are questions regarding editing the way the language model behaves and assessing compliance with changes. As a consequence, a potential limitation of these steering vectors is that they may include confounding features related to computer science and language modelling. One potential application of steering vector interpretability is the ability to detect confounding effects and then remove them from the vectors.

A.2 Sparse autoencoders

We use the SAEs in Gemma Scope [9]. Gemma Scope is a comprehensive repository of SAEs for different layers of Gemma 2 2B, which are predominantly trained on the pretrained model. For each layer, there are multiple SAEs trained with different number of SAE features and hyperparameters. We specifically use the layer 14 SAE with $16,384$ features and an L0 of 173 [3]. The L0 statistic measures the mean number of active features.

SAEs in Gemma Scope are trained with the JumpReLU architecture [15]. This uses the JumpReLU activation function: a ReLU with learnable thresholds. The effect of this is that all feature thresholds are positive (min of 1.12 and max of 9.86). Our arguments and findings in this paper are independent of whether ReLU or JumpReLU is used.

B Comparing the corrigibility steering vectors at different layers

Our analysis uses the layer 14 corrigibility steering vector because we found this to be the best layer to steer model behaviour at. To enable comparison of steering vectors at different layers, we use the definition of *steerability* proposed in [18]. A number of steps are required to build this metric. First, a model can be thought of as have a propensity to exhibit a certain behaviour. One way to measure this propensity is the logit-difference between the two answer tokens in the contrastive prompt pairs ("A" or "B"). One of these tokens will encode the behaviour of interest and the other will not, therefore, a measure of propensity through logit-difference is

$$m_{LD} = \text{Logit}(y_+) - \text{Logit}(y_-). \tag{4}$$

We use the average m_{LD} across a held-out portion of the contrastive prompt pairs dataset (the test tranche) to get a measure of model propensity. Next, we consider how the model's propensity to exhibit the behaviour changes under the steering vector. The steering vector is added under a range of multipliers λ, and, for each multiplier, the model propensity is calculated. We calculate logit-difference propensity for all $\lambda \in \{-1.5, -1.0, -0.5, 0.0, 0.5, 1.0, 1.5\}$. The resulting propensity values are referred to as the *propensity curve* [18, 13]. We might expect that this curve is monotonically

[3] See `https://huggingface.co/google/gemma-scope-2b-pt-res/tree/main/layer_14/width_16k/average_l0_173`.

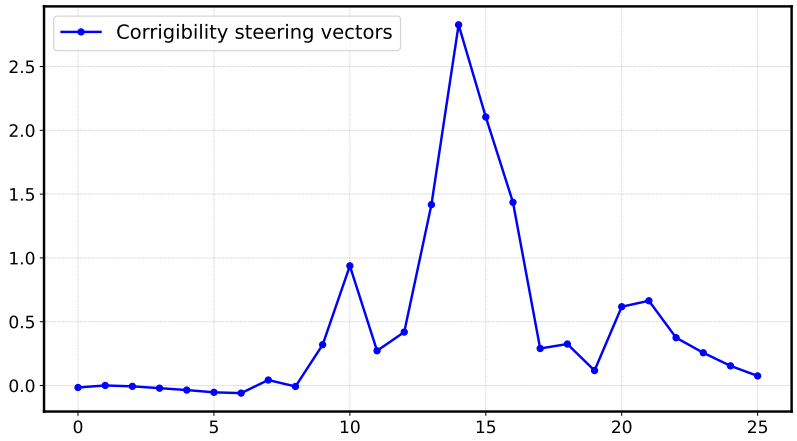

Figure 4: **The corrigibility steering vector extracted at layer 14 has the highest steerability.** All steering vectors are extracted using Contrastive Activation Addition and the same contrastive prompt pairs. Steerability is defined as in [18].

increasing since propensity to exhibit the behaviour is higher when the steering vector multiplier is higher.

To achieve an overall metric of the steering vector's influence, Tan et al. [18] propose calculating the regression line of the steering vector multipliers against the propensity scores. They then define *steerability* as the slope of this line. More complete details on defining *steerability* can be found in [18].

Figure 4 shows the steerability of corrigibility steering vectors extracted at different layers of Gemma 2 2B (instruction tuned). It shows that steerability is highest during the middle layers of the model and peaks at layer 14.

C Decomposing steering vectors for other behaviours

This section reports results for other behaviours, using the behaviours in the original Contrastive Activation Addition paper: sycophancy, survival-instinct, coordinate-other-AIs, corrigible-neutral-HHH, myopic-reward, refusal and hallucination [13]. These datasets are largely originally sourced from Perez et al. [14].

C.1 Steering vectors are out-of-distribution

Figure 5 shows the L_2-norms of the layer 14 steering vectors against the distribution of L_2-norms for model activations at that layer. The figure shows that all seven steering vectors have norms outside the distribution, implying that the SAE encoder bias vector will consistently play a disproportionate role during direct SAE-decomposition.

Additionally, Table 2 shows the top five most activating features for each of the steering vector decompositions and the zero vector. All decompositions are extremely similar. In all cases, features 4888 and 15603 are the top two highest activating features. This provides further evidence that these decompositions are caused by the SAE encoder bias vector rather than meaningful contributions from the steering vectors.

C.2 SAEs only permit decompositions with non-negative reconstruction coefficients

Steering vectors can have meaningful negative projections in SAE feature directions which makes SAE-decomposition misleading. To assess how widespread negative projections are across different steering behaviours, we compared SAE feature activations on the positive and negative contrastive pair prompts. Given steering vectors are trained by subtracting activations on contrastive prompt pairs, we would expect that the difference in feature activations is somewhat indicative of the steering

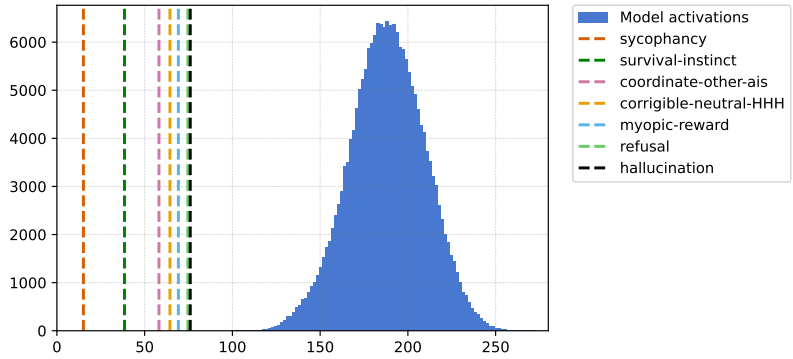

Figure 5: **Steering vector L_2-norms.** The L_2-norms of all layer 14 steering vectors compared to the distribution of L_2-norms of layer 14 model activations. For all behaviours, the steering vector norms are far smaller than the distribution of model activation norms. Model activations are taken over random sequences in The Pile [6], totalling 200,000 tokens.

Zero vector		Sycophancy steering vector		Coordinate-other-ais steering vector		Corrigible-neutral -HHH steering vector	
Feature	Activation	Feature	Activation	Feature	Activation	Feature	Activation
4888	89.06	4888	90.90	4888	90.59	4888	95.04
15603	35.94	15603	36.69	15603	34.35	15603	36.34
7589	19.80	7589	19.32	7589	20.11	12695	22.64
15471	11.84	15471	12.38	9956	12.24	7589	18.89
2350	10.74	2350	10.52	15471	11.18	2350	11.35

Hallucination steering vector		Myopic-reward steering vector		Refusal steering vector		Survival-instinct steering vector	
Feature	Activation	Feature	Activation	Feature	Activation	Feature	Activation
4888	81.73	4888	99.70	4888	101.77	4888	91.37
15603	32.63	15603	41.11	15603	41.35	15603	36.46
7589	21.84	4107	28.95	7655	16.05	7589	19.59
8841	12.27	7589	22.33	7589	14.12	15471	11.82
9956	11.87	15471	15.19	10520	13.15	2350	11.56

Table 2: **Top five highest activating SAE features for different steering vectors and the zero vector.** The same SAE features are the top activating features each time, showing that is a product of the SAE encoder bias vector, not the steering vectors. All steering vectors extracted at layer 14.

vector. If a steering vector has a meaningful positive projection in a certain feature direction, this will materialise higher activations on the positive contrastive prompts relative to the negative contrastive prompts. Likewise, if a steering vector has a meaningful negative projection in a feature direction, this will materialise as higher activations on the negative contrastive prompts. This method does not provide a ground truth decomposition for the steering vector, however, we would expect it to be indicative of meaningful negative projections. It avoids directly decomposing the steering vector, which, as Section 3.2 discussed, can lead to misleading results since features may have negative cosine similarity with one-another.

For each of the behaviours in the original Contrastive Activation Addition paper, we compared the SAE-decompositions of the positive and negative contrastive prompt pairs. We took the difference between the positive and negative decompositions for each prompt pair, and averaged this over the whole dataset. To consider the impact of negativity, we considered the 100 features with the largest

magnitude (i.e. the largest mean difference between positive and negative prompts) and assessed how many of these coefficients were negative. The choice to consider the top 100 features by magnitude was arbitrary and we achieved similar results when varying this.

Behaviour	**Number of negative features**
sycophancy	56
coordinate-other-ais	50
corrigible-neutral-HHH	58
hallucination	55
myopic-reward	51
refusal	47
survival-instinct	44

Table 3: **Behaviour and number of negative features**. Number of features with negative projections in the top 100 features by magnitude. All steering vectors extracted at layer 14.

Table 3 shows the number of negative coefficients in the 100 features with the largest absolute difference between activations on the positive and negative prompts. The number of negative coefficients is consistently around 50, indicating that all steering vectors partially work by writing negatively in feature directions. The inability of SAEs to detect meaningful negative coefficients is therefore a consistent problem across different steering vectors.

D Is a global interpretation of steering vectors possible?

An interesting question to consider is whether a global interpretation of steering vector is actually possible. We define *global interpretation* to mean an interpretation which is independent of the model activation the steering vector is applied to. In contrast, a *local interpretation* would be an interpretation which is only applicable when the steering vector is added to a subset of model activation with particular characteristics. Section 3.1 showed that steering vectors can have meaningful negative projections in feature directions and argued that one reason this made decomposing steering vectors challenging was that it is difficult to separate negative projections in one feature direction from positive projections in a feature direction with negative cosine similarity.

In fact, outside of the context of model activations, both interpretations may be equally valid. Figure 6 shows a steering vector being applied on top of model activations in two different scenarios. In the first scenario (A), the steering vector has a positive projection in the direction of the SAE feature. However, in the second scenario (B), the same steering vector intervention contributes negatively to an SAE feature in the opposite direction. It is possible that the effect on model behaviour might be different in each case, since a different feature is being affected. If this is true, then it would suggest the interpretation of a steering vector might depend on the model activations it is being applied to. Critically, the effect and interpretability of a steering vector may differ significant when applied to model activations different from those it was extracted from. A more complete exploration of this effect is left for future research.

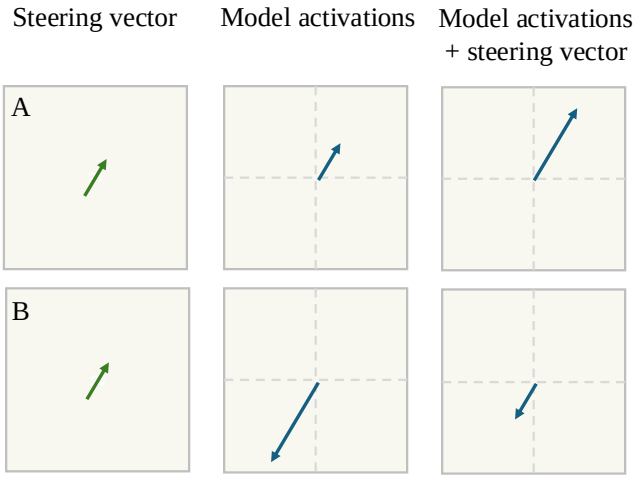

Figure 6: **Illustration of why steering vector interpretability may depend on the model activations the vector is added to**. Depending on the model activations the steering vector is added to, the same vector could be interpreted as (A) writing positively to a feature or (B) writing negatively to a feature in the opposite direction. These two scenarios cannot be separated out-of-context.